

Lecture: 0110

Registers, Counters & State Machines

Building Memory That Does Something

Chain flip-flops together and a circuit can store words, count, and follow a plan.

CSA222: Modern Computer Architecture

One Flip-Flop = One Bit

L5 gave us a reliable 1-bit memory. Today we build things worth remembering.



The D flip-flop

Captures one bit on the clock edge, holds it for the cycle (L5).



Edge-triggered & safe

Master-slave design blocks data races — predictable updates.



Timing rules

Setup, hold, metastability — the constraints that keep it working (L5).



But just ONE bit...

A CPU needs 32-bit words, counters, and control. So we chain them.

The jump from 'store a bit' to 'store a word, count events, run a program' is what today is about.

Today's Roadmap

Three structures, each more capable than the last.

1

Registers

n flip-flops in parallel = an n-bit word store. Add shifting for data movement.



2

Counters

Flip-flops that count clock pulses — the basis of timers and addresses.



3

State Machines

The general model of any sequential circuit: states, transitions, outputs.

The big idea: an FSM is the universal blueprint — registers and counters are just special cases of it. It's how a CPU 'thinks'.



when studying
computer engineering
goes from this...



The Flip-Flop Toolkit

Four flip-flops, four truth tables — and the one reverse-lookup table that makes design possible.

PRIMARY DEFINITION

A flip-flop is a one-bit memory that updates only on a clock edge. Change what feeds its input and you change what it does.

SR Set / Reset — the original latch

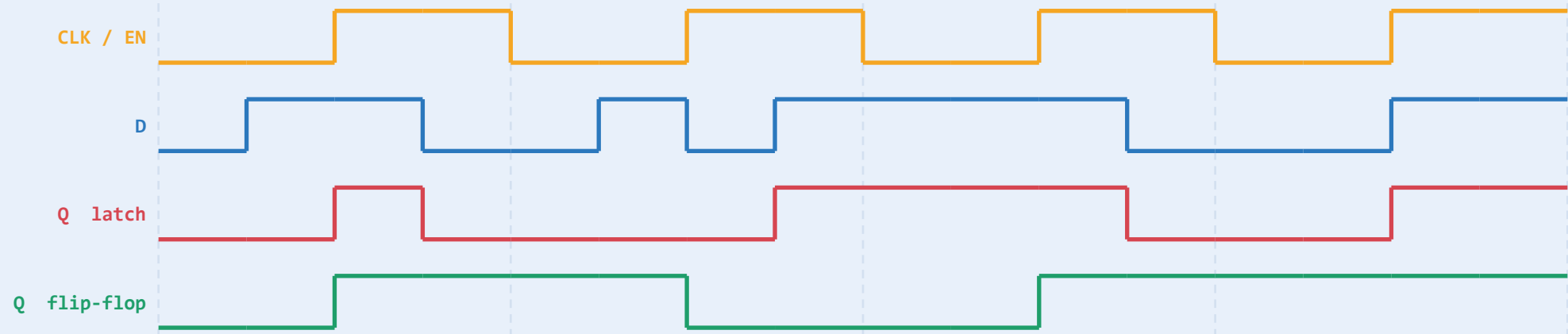
D Delay — copies its input, one cycle later

JK The universal one — SR with toggle

T Toggle — flips on command. Counters live here.

Latch vs Flip-Flop

Both remember a bit. The difference is WHEN they are allowed to change.



Same clock, same data — two very different outputs. The latch copies D the whole time EN is high; the flip-flop looks once, at the edge.

LEVEL-TRIGGERED — LATCH

Transparent: while the enable is high, Q follows D continuously. Any wobble on D passes straight through.

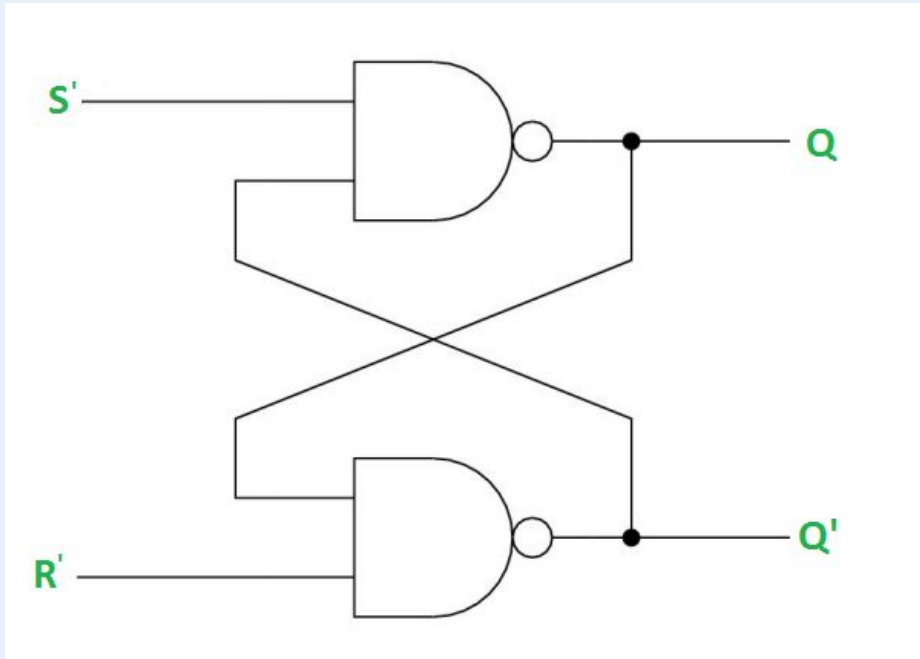
EDGE-TRIGGERED — FLIP-FLOP

Q samples D at one instant only, then holds for the whole cycle. This is why we can reason about timing at all.

Everything from here on uses edge-triggered flip-flops. When this lecture says 'flip-flop', assume the rising clock edge.

The SR Latch — Where Memory Starts

Two NOR gates feeding each other. The simplest circuit that can remember.



CHARACTERISTIC TABLE

S	R	Q^+	MEANING
0	0	Q	Hold — remembers the last value
0	1	0	Reset the bit to 0
1	0	1	Set the bit to 1
1	1	—	Forbidden — both outputs go 0

THE FLAW

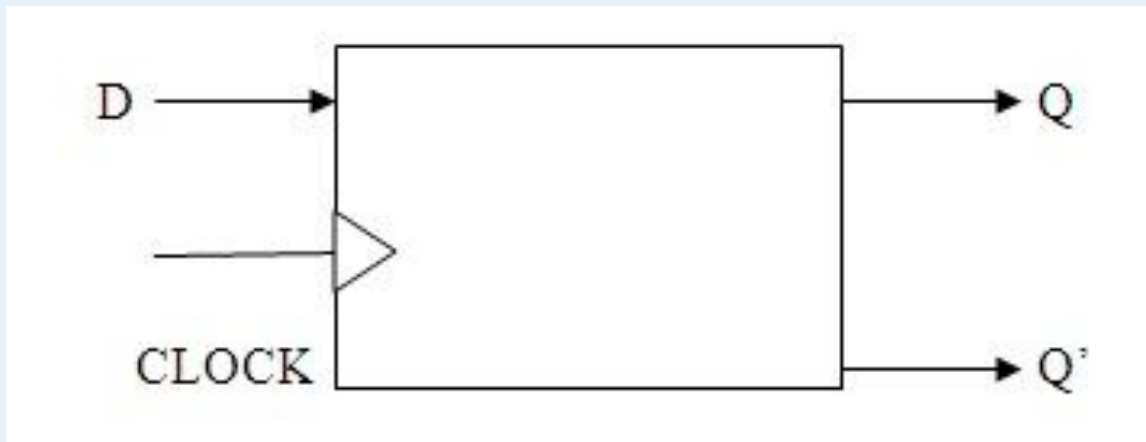
$S = R = 1$ breaks the promise that Q and Q' are opposites — and when both inputs drop together the result is unpredictable.

Fixing that forbidden row is exactly what the JK flip-flop is for — and the fix turns out to give us the toggle we need for counters.

The D Flip-Flop — One Input, No Surprises

Tie S and R together through an inverter and the forbidden state becomes unreachable.

SYMBOL & PIN NAMES



CHARACTERISTIC TABLE

Q	D	Q(n+1)	WHAT HAPPENED
0	0	0	stayed at 0
0	1	1	loaded a 1
1	0	0	loaded a 0
1	1	1	stayed at 1

CHARACTERISTIC EQUATION

$Q(n+1) = D$ — the next state is simply whatever D was at the edge.

WHY IT IS THE DEFAULT

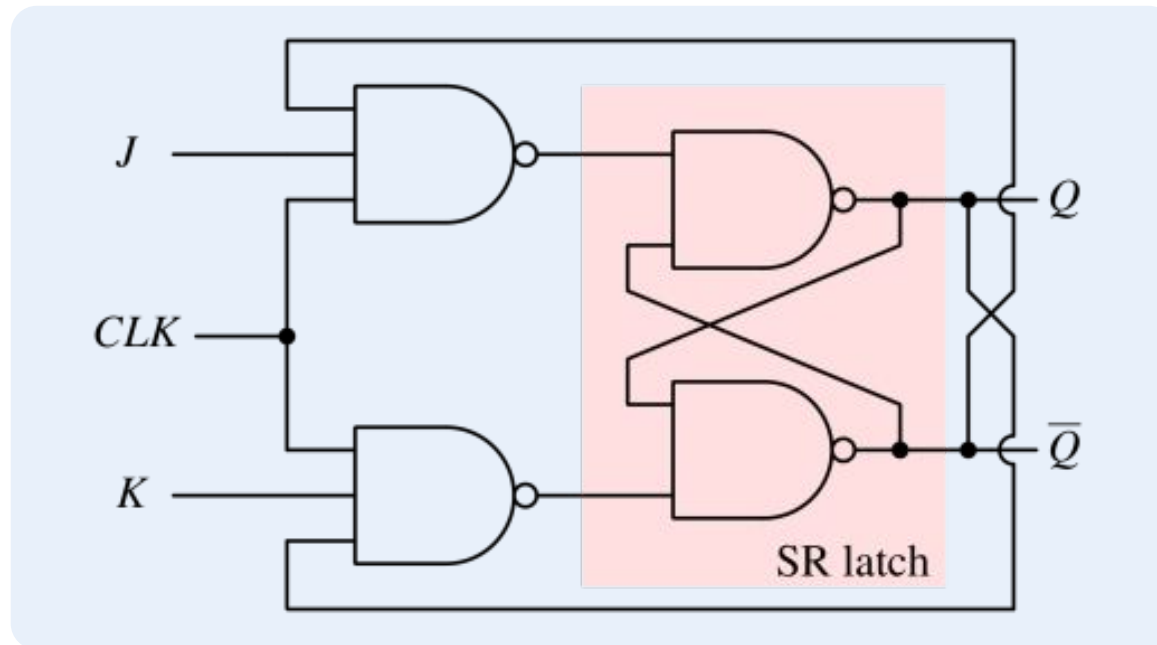
One data input means one thing can go wrong instead of four. Every register, pipeline stage and state register in a modern CPU is built from D flip-flops.

THE COST

A D flip-flop cannot toggle by itself — it only copies. To make it flip, you must feed it the opposite of its own output. Hold that thought.

The JK Latch — Reclaiming the Fourth Row

Same four input combinations as SR, but the illegal one is redefined as 'toggle'.



CHARACTERISTIC TABLE

J	K	Q^+	MEANING
0	0	Q	Hold — no change
0	1	0	Reset to 0
1	0	1	Set to 1
1	1	Q'	TOGGLE — flip to the opposite

CHARACTERISTIC EQUATION

$$Q(n+1) = J\bar{Q}(n) + Q\bar{K}(n)$$

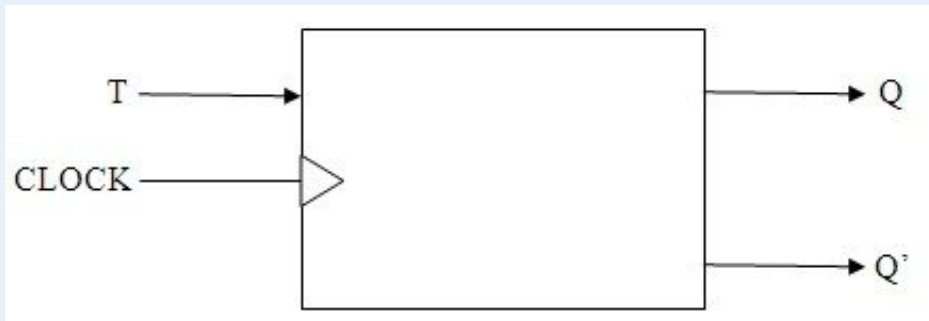
WHY THIS MATTERS TODAY

The JK flip-flop can do everything SR and D can do, plus one new trick: toggle. Toggling is counting. Tie J and K together and you get the flip-flop the rest of this lecture depends on.

The T Flip-Flop — The One Counters Are Made Of

A single input with a single job: $T = 1$ means 'flip', $T = 0$ means 'stay'.

SYMBOL



CHARACTERISTIC TABLE

T	Q	Q ⁺	ACTION
0	0	0	hold
0	1	1	hold
1	0	1	flip
1	1	0	flip

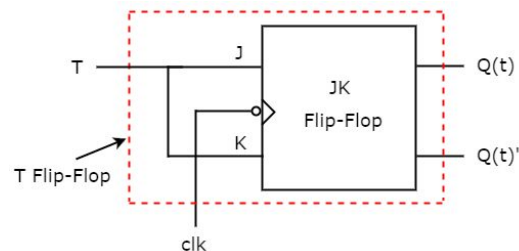
Remember this device. Every counter in the next two sections is a row of T flip-flops with different logic deciding when each one flips.

CHARACTERISTIC EQUATION

$$Q(n+1) = TQn' + T'Qn = T \text{ XOR } Qn$$

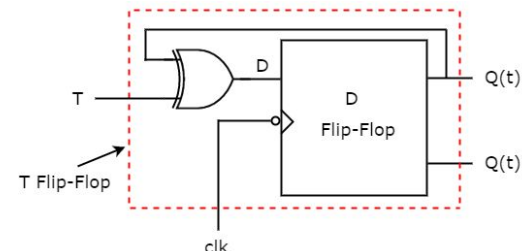
TWO WAYS TO BUILD ONE

1 • From a JK flip-flop



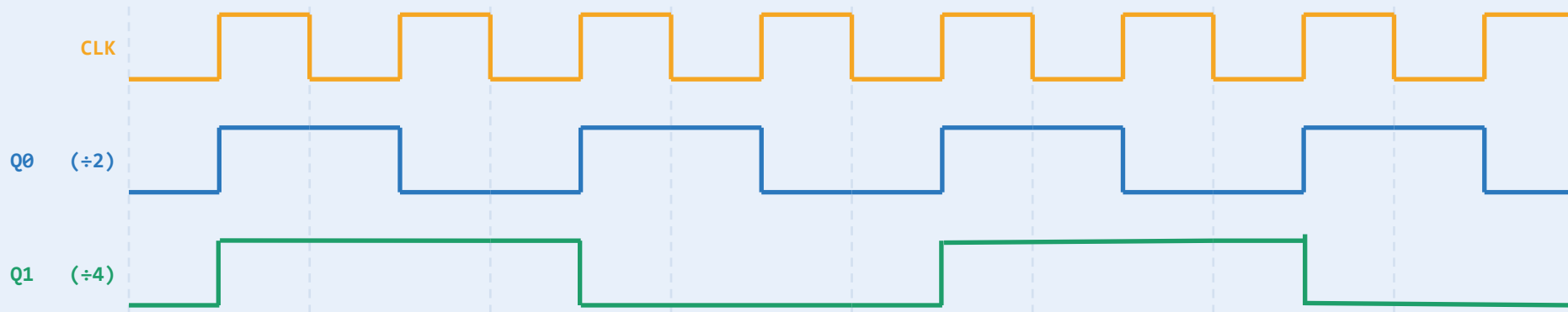
Tie J and K to the same wire.

2 • From a D flip-flop



Hold T at 1 and You Get a Divide-by-2

A permanently toggling flip-flop halves the clock frequency. Chain them and you have a counter.



Q0 completes one cycle for every two clock cycles. Q1 is clocked by Q0, so it completes one cycle for every four. Each extra flip-flop halves the frequency again — and the bits you are looking at spell out 0, 1, 2, 3 in binary.

AS A DIVIDER

n stages divide the clock by 2^n . This is how a 32 768 Hz watch crystal becomes a 1 Hz tick.

AS A COUNTER

Read the stages as bits and the same circuit is counting up in binary — same wires, different reading.

THE LINK

Counting and dividing are the same operation. That is why one circuit serves timers, PWM and program counters.

The Four Flip-Flops at a Glance

One table to keep. Everything else in this lecture is built from these four rows.

TYPE	INPUTS	WHAT IT DOES	CHARACTERISTIC EQ.	TYPICALLY USED FOR
SR	S, R	Set, reset or hold — but $S = R = 1$ is illegal	$Q^* = S + R' \cdot Q$	Latches, simple control
D	D	Copies its input to the output on the edge	$Q^* = D$	Registers, pipeline stages, state registers
JK	J, K	Set, reset, hold — and toggle when both are 1	$Q^* = J \cdot Q' + K' \cdot Q$	General-purpose sequential design
T	T	Flips when $T = 1$, holds when $T = 0$	$Q^* = T \text{ XOR } Q$	Counters and frequency dividers

THEY ARE ALL THE SAME DEVICE

Every one of these is an SR latch pair with different logic bolted onto the front. Choosing a flip-flop type is really choosing how much of the next-state logic you want built in already.

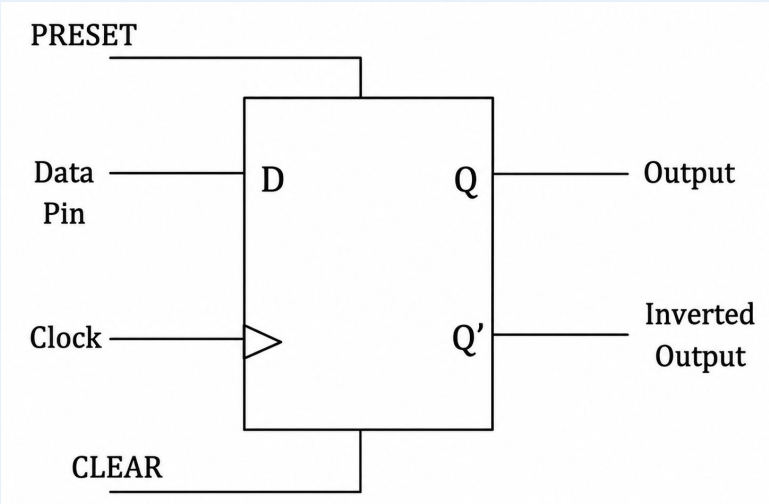
WHAT REAL HARDWARE USES

Modern chips and every FPGA use D flip-flops almost exclusively — they are the smallest and fastest. JK and T are design conveniences: you reason in T, the synthesis tool builds it out of D and an XOR.

Preset, Clear and Reset

Every real flip-flop has a back door that ignores the clock. Counters and state machines both need it.

THE EXTRA PINS



	ASYNCHRONOUS	SYNCHRONOUS
When it acts	Immediately	On the next clock edge
Needs the clock?	No	Yes
Timing risk	Can violate setup/hold	None — it is ordinary logic
Typical use	Power-on reset	Mod-N counter wrap, FSM restart

Rule of thumb: one asynchronous reset for power-on, synchronous resets for everything the design does while it is running.

WITHOUT A RESET

Flip-flops power up in a random state. A counter might start at 173, and a state machine might start in a state that has no way out. The circuit works perfectly in simulation and hangs on the bench.

WITH A RESET

One known starting point, guaranteed. Every FSM you draw from now on gets a reset arrow pointing at its initial state — no exceptions.

1

Registers

The foundational building blocks of CPU storage.

PRIMARY DEFINITION

A register is just a row of flip-flops sharing a clock — the **fastest, closest memory** a CPU has.



Flip-Flop Row
Multi-bit storage



Shared Clock
Atomic updates



Fastest Speed
Closest to CPU

From One Bit to Many

Put flip-flops side by side, share a clock, and you store a whole word at once.

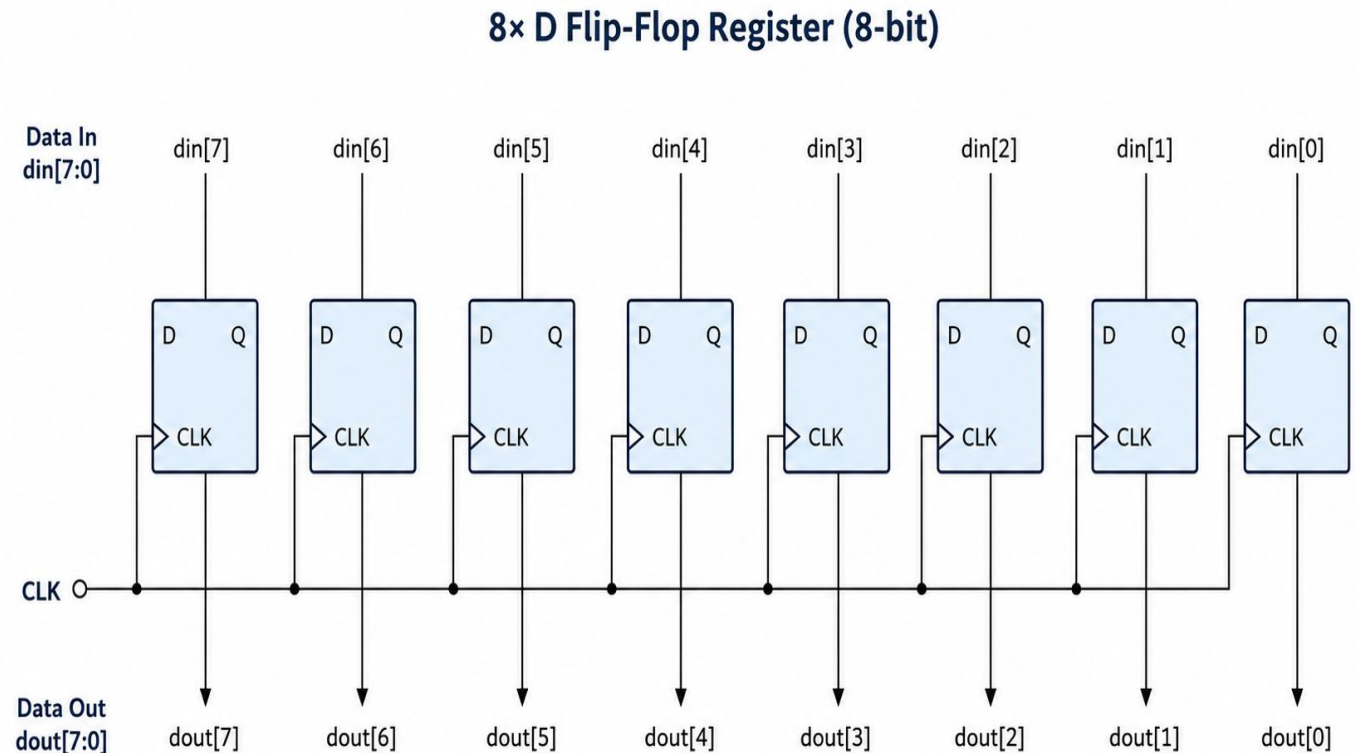
Atomic Multi-Bit Storage

*All four bits are captured on the **SAME clock edge** — the word updates atomically, as one single unit.*

THE SYSTEM SECRET

What is a CPU register?

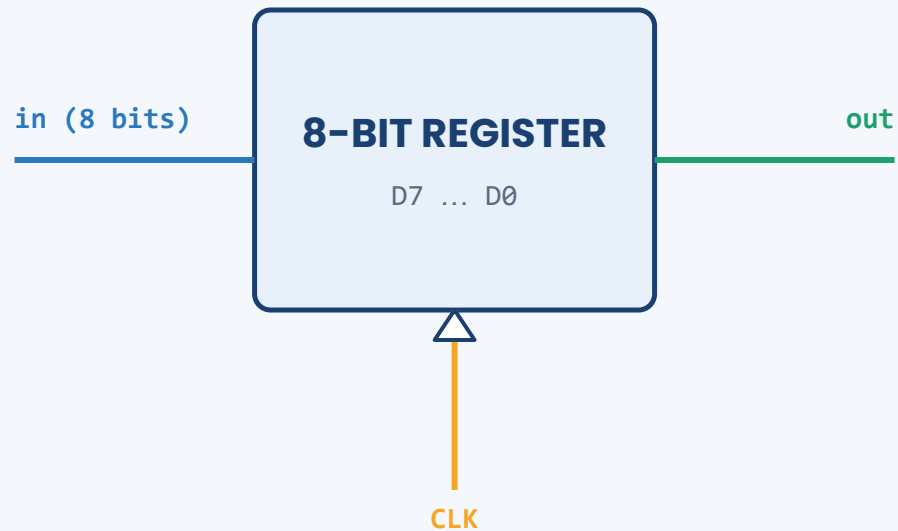
This is literally what a CPU register is: **32 or 64 D-flip-flops** sharing a single, synchronous clock. That is the entire secret.



The Register

Store, hold, and read back an n-bit value — the CPU's working scratchpad.

BLOCK DIAGRAM



WHERE REGISTERS LIVE

The register file — 32 registers in MIPS/RISC-V — is a CPU's fastest storage.

It serves as the immediate staging area for active operations, feeding data directly to execution units.

Also: the program counter (PC), instruction register, and pipeline stage registers are all just registers.

Faster than cache, faster than RAM — because it's flip-flops right next to the ALU.

The Load Enable

A register must HOLD its value across clock edges until you choose to update it.

THE PROBLEM

A plain flip-flop grabs D on EVERY clock edge. But a register should keep its value for many cycles, updating only when told.

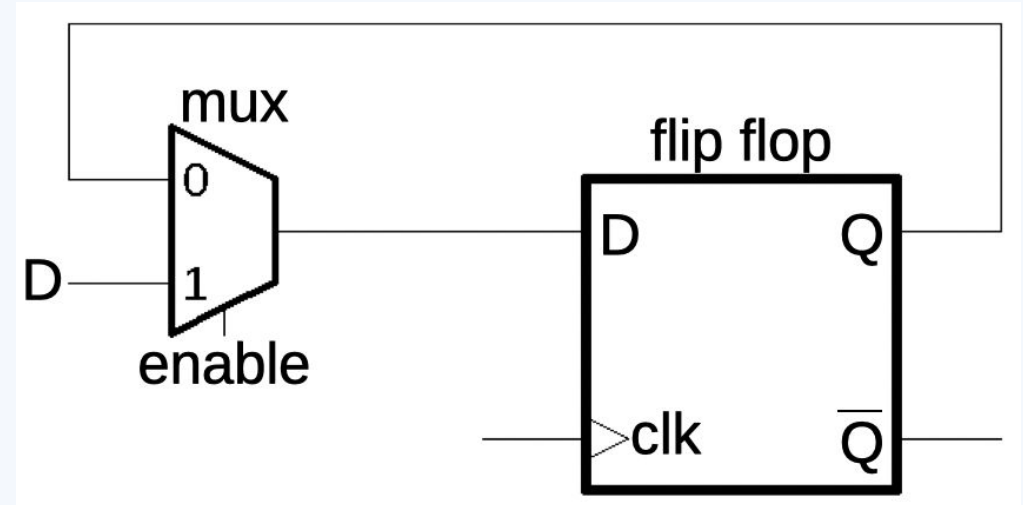
THE FIX — a load-enable MUX per bit:

LOAD = 1 → feed new data in.

LOAD = 0 → feed the flip-flop's OWN output back in (hold).

A 2:1 MUX on each bit's D input chooses between 'new value' and 'keep current'.

ONE BIT'S PATH

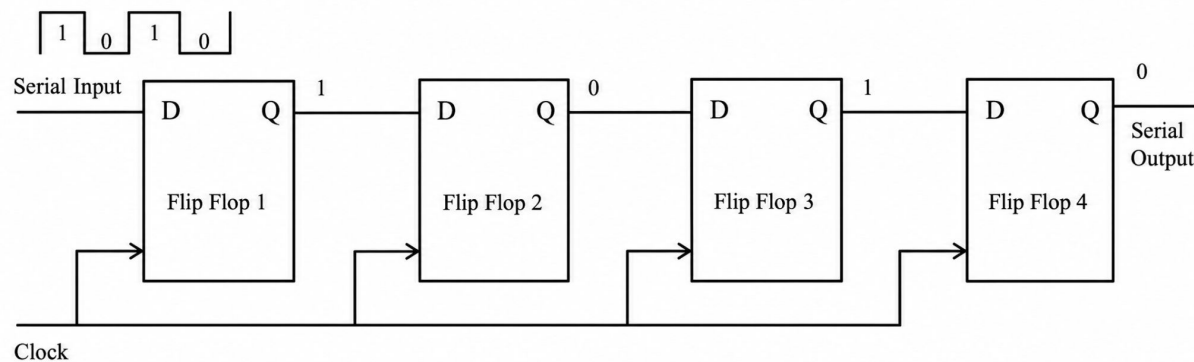


The MUX + feedback loop = a register that remembers until LOAD says otherwise.

The Shift Register

Chain flip-flops output-to-input: each clock edge, every bit shifts one place.

BLOCK DIAGRAM



HOW IT SHIFTS

Each flip-flop's output feeds the NEXT one's input. A bit entered at 'in' walks right, one FF per clock.

It's a conveyor belt for bits — the basis of serial data transfer and multiply/divide-by-2 operations.

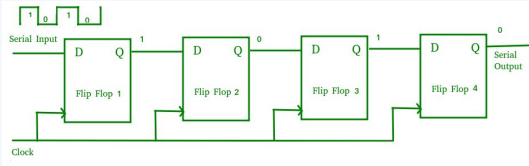
Four Shift Register Types

Named by how data goes IN and comes OUT: serial or parallel.

SISO

Serial In, Serial Out

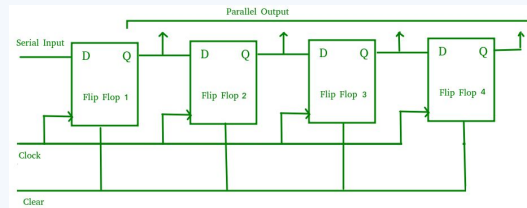
One bit in, one bit out per clock — a pure delay line.



SIPO

Serial In, Parallel Out

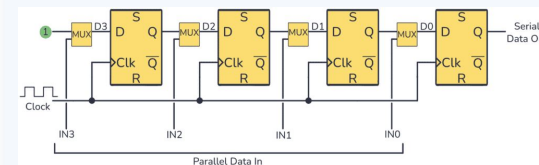
Bits enter one at a time, read out all at once — serial→parallel converter.



PISO

Parallel In, Serial Out

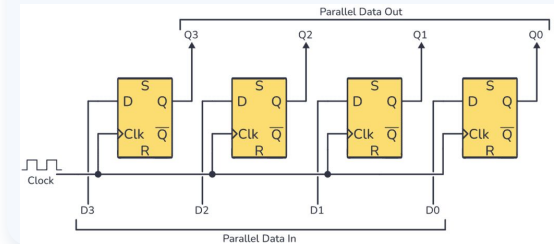
Load a whole word, shift it out bit by bit — parallel→serial converter.



PIPO

Parallel In, Parallel Out

Load and read a full word — this is just a normal register with load.



SIPO and PISO are the workhorses: they convert between serial links (few wires) and parallel buses (fast).

What Shift Registers Do

Small circuit, surprisingly many jobs.



Serial ↔ parallel

UART, SPI, USB all send data one bit at a time over few wires; shift registers reassemble it into bytes.



Multiply / divide by 2

Shifting binary left multiplies by 2; shifting right divides by 2. Free arithmetic, no adder.



Delay lines & buffers

A SISO register delays a signal by exactly N clock cycles — used in DSP and pipelines.



Pseudo-random (LFSR)

Add feedback via XOR and a shift register generates long pseudo-random sequences — used in crypto & test.

SECTION 02

2. Counters

The Beating Clock of Digital Systems

Flip-flops configured to count clock pulses, serving as the essential foundation for timing, addressing, and control flow.

CORE APPLICATIONS

**Timers & Clock Dividers**

Measuring precise time intervals and scaling frequencies.

**Address Generation**

Stepping through memory locations sequentially.

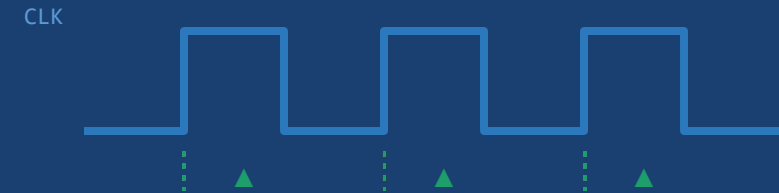
**The Program Counter (PC)**

The engine driving instruction fetch and CPU execution flow.

VISUALIZING CLOCK & COUNT

How a Counter Advances

Each rising edge of the clock trigger increments the register state.

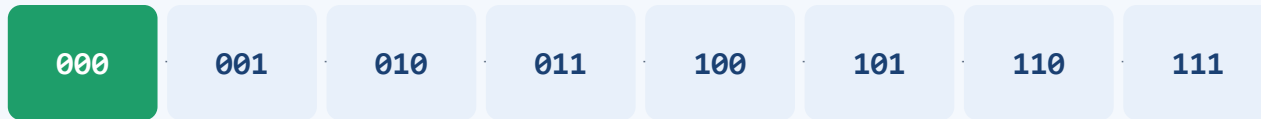


STATE TRANSITION (N-BIT OVERFLOW)

What Is a Counter?

A register whose value advances by 1 on every clock pulse — it counts.

A 3-BIT COUNTER CYCLES THROUGH 0 → 7 → 0...



↻ wraps back to 000

Each clock pulse = +1.

After the max value (111 = 7), it rolls over to 000 — exactly like an odometer, or 8-bit overflow from L4.

An n -bit counter counts 0 to 2^n-1 , then wraps. It's a register + an incrementer + feedback.

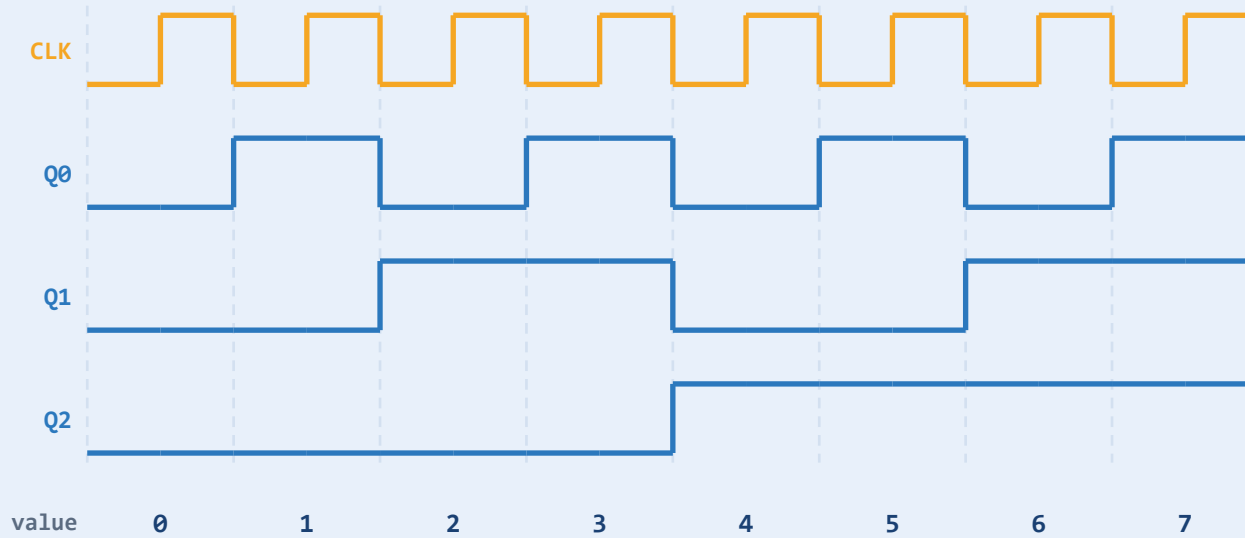
WHY COUNTERS MATTER

The Program Counter (PC) is a counter — it holds the address of the next instruction and increments each cycle.

Also: timers, clock dividers, memory-refresh logic, event counters — all built on this.

Reading the Count as Waveforms

Look at the bits over time and the pattern that makes counters buildable jumps out.



Q0 changes every clock. Q1 changes every second clock. Q2 changes every fourth.

THE RULE BEHIND IT

A bit flips exactly when every bit below it is 1.

Q0 flips always. Q1 flips when Q0 = 1. Q2 flips when Q1·Q0 = 1.

IN T FLIP-FLOP TERMS

$T0 = 1$

$T1 = Q0$

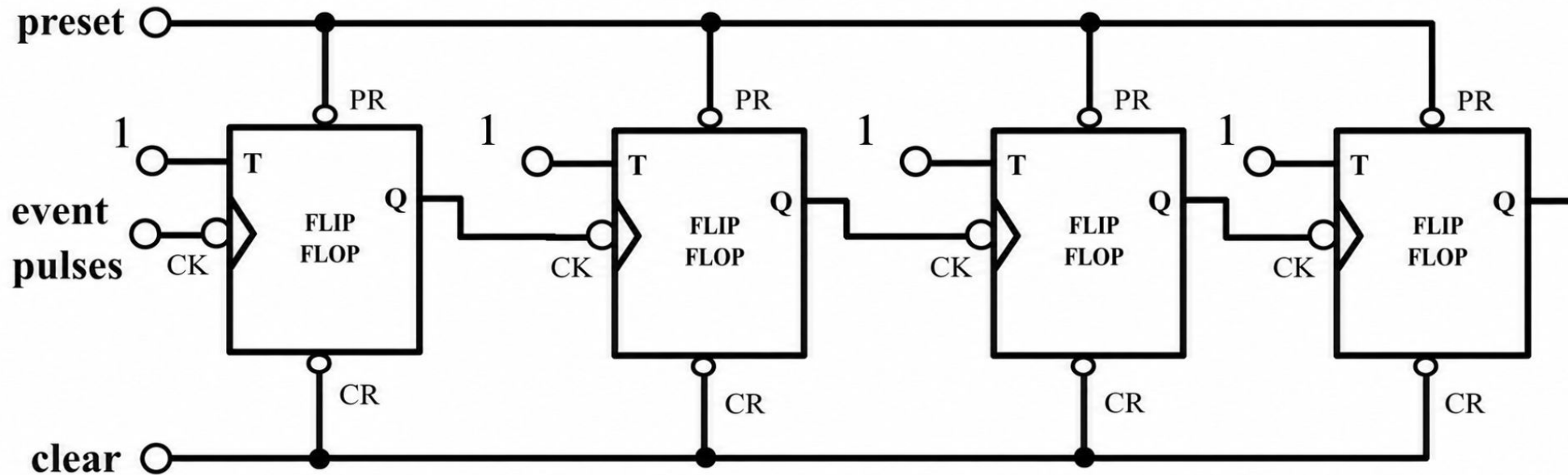
$T2 = Q0 \cdot Q1$

THAT IS THE WHOLE DESIGN

Nothing else is needed for a plain binary up-counter. Two different circuits implement it — and choosing between them is the next four slides.

Ripple (Asynchronous) Counter

Each flip-flop's output clocks the next — simple, but the carry ripples.



Only FF0 sees the real clock; each next FF is clocked by the previous output — the change 'ripples' along.

⚠ The ripple delay means the count is briefly **WRONG** while it propagates — a problem at high speed.

Why Ripple Counters Break at Speed

Follow the count from 7 to 8 one gate delay at a time, and watch three wrong numbers appear on the pins.

TIME AFTER EDGE	Q3	Q2	Q1	Q0	VALUE ON PINS
before	0	1	1	1	7 — correct
1 × t _{pd}	0	1	1	0	6 — wrong
2 × t _{pd}	0	1	0	0	4 — wrong
3 × t _{pd}	0	0	0	0	0 — wrong
4 × t _{pd}	1	0	0	0	8 — correct at last

Anything reading those pins during the ripple — a comparator, a decoder, a memory address bus — sees 6, then 4, then 0.

THE SPEED LIMIT

The count is only valid after ALL n stages have settled:

$$t_{\text{settle}} = n \times t_{\text{pd}}$$

$$f_{\text{max}} = 1 / (n \times t_{\text{pd}})$$

PUT NUMBERS ON IT

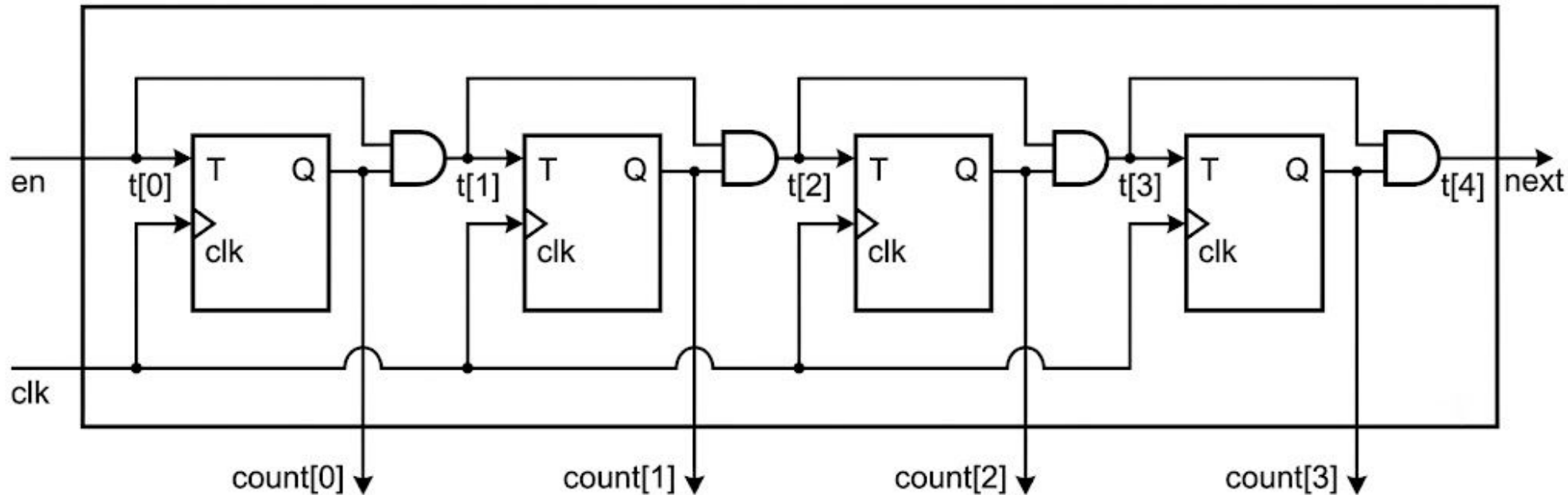
At $t_{\text{pd}} = 10 \text{ ns}$, a 4-bit ripple counter tops out near 25 MHz — and a 16-bit one near 6 MHz. A modern CPU clock is 3 GHz.

THE WAY OUT

Give every flip-flop the same clock, and compute in advance which ones should flip.

Synchronous Counter

All flip-flops share ONE clock — they all update at the exact same instant.



How It Works

Combinational logic (the & gates) computes each FF's next bit; the shared clock updates them all together.

Key Advantage

✓ No ripple delay — the whole count is correct the instant after the edge. This is what real designs use.

Ripple vs Synchronous

Same count sequence, very different timing behaviour.

	RIPPLE (async)	SYNCHRONOUS
Clocking	Each FF clocks the next	All FFs share one clock
Speed	Slow — carry ripples through	Fast — all update at once
Glitches	Yes — transient wrong counts	No — clean output
Complexity	Simple, fewer gates	Needs next-state logic
Real use	Toy / low-speed dividers	Everything performance-critical

Trade-off in one line: ripple is cheaper to build; synchronous is the only one safe at real clock speeds.

Mod-N & Divide-by Counters

Reset early to count any range — or use a counter to slow a clock down.

MOD-N: STOP EARLY

A 3-bit counter naturally counts mod-8 (0–7). To count mod-6 (0–5), detect the value 6 and reset to 0.



↻ reset at 6

Used for: clock dividers, digital clocks (mod-60 for seconds!), any custom cycle length.

DIVIDE-BY: SLOW THE CLOCK

Each flip-flop in a counter toggles at HALF the rate of the one before it.

So bit 0 = clock ÷ 2, bit 1 = clock ÷ 4, bit 2 = clock ÷ 8...

A counter is a frequency divider for free. This is how a 32 kHz crystal becomes a 1 Hz tick in a watch.

Counters in the Wild

Once you see them, they're everywhere.



Program Counter

Holds the address of the next instruction; increments each cycle, or jumps on a branch. The heart of instruction flow.



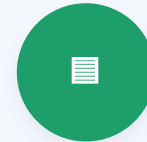
Timers & delays

Count clock ticks to measure time — every microcontroller timer, watchdog, and PWM generator is a counter.



Frequency division

Turn a fast crystal oscillator into slower usable clocks — watches, UART baud rates, display refresh.



Memory addressing

Sequentially step through addresses in DMA transfers, refresh cycles, and scan-out logic.

3. Finite State Machines

The universal model of sequential logic: a set of states, rules to move between them, and outputs. This is how a CPU 'thinks'.

What Is a Finite State Machine?

A circuit that is always in exactly ONE of a finite set of states, and moves between them on rules.

THREE INGREDIENTS

1

States

A finite list of situations the machine can be in (e.g. IDLE, RUNNING, DONE).

2

Transitions

Rules for moving between states, based on current state + inputs.

3

Outputs

What the machine produces in each state (or during each transition).

EVERYDAY EXAMPLE

A traffic light is an FSM.

States: GREEN, YELLOW, RED.

Transitions: timed (green→yellow→red→green).

Outputs: which lamp is lit.

A vending machine, an elevator, a game character's AI, a network protocol — all FSMs. This model is everywhere.

Two Flavours: Moore vs Mealy

The difference is simply WHERE the output comes from.

MOORE MACHINE

Output depends on the STATE only

The output is written inside each state circle

It changes only when the state changes

Simpler to reason about, and glitch-free

May need more states to do the same job

Output appears one cycle after the input

MEALY MACHINE

Output depends on STATE + INPUT

The output is written on each transition arrow

It can react to an input within the same cycle

Often needs fewer states

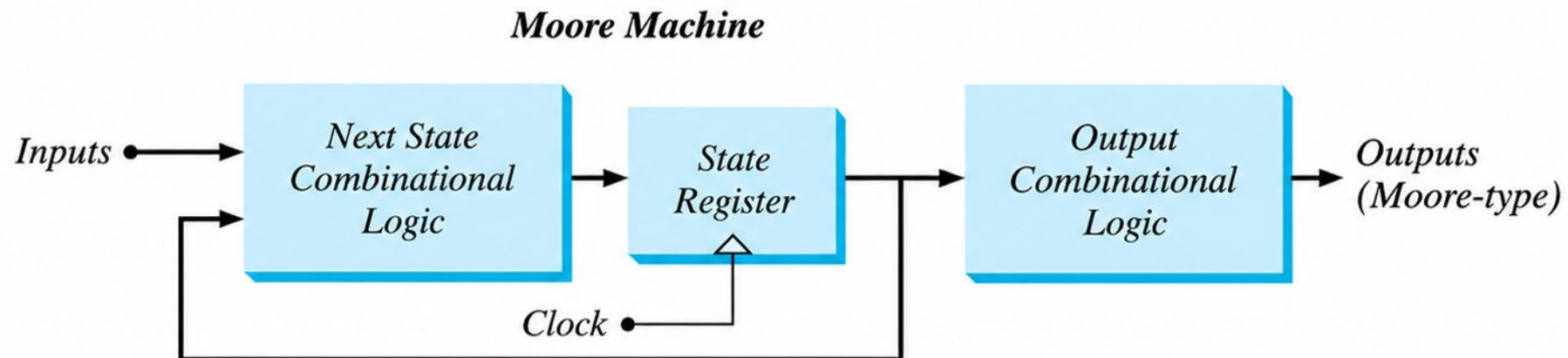
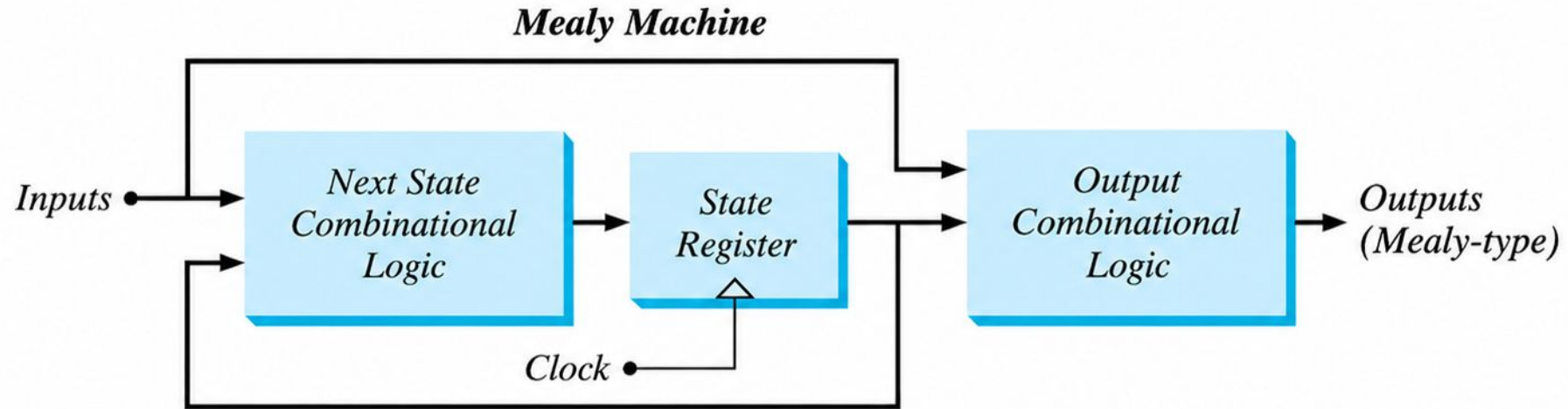
Output can glitch when the input glitches

Output appears in the same cycle as the input

Rule of thumb: Moore = 'output in the circle', Mealy = 'output on the arrow'. Both compute the same things.

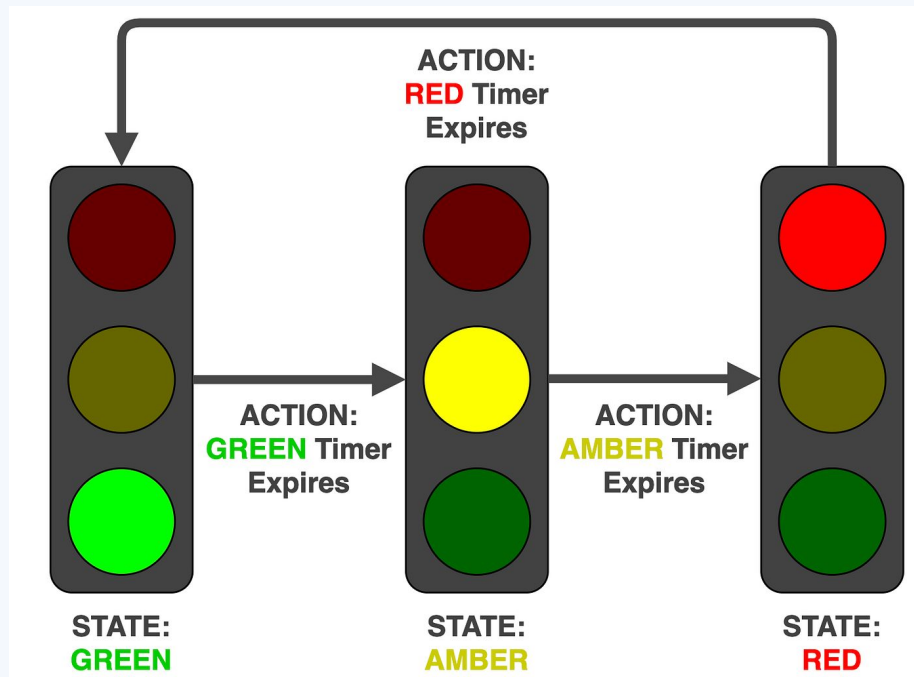
The Anatomy of Any FSM

Every sequential circuit is this same three-part structure.



The State Diagram

The picture that captures an FSM: circles are states, arrows are transitions.



HOW TO READ IT

Each circle = one state.

Each arrow = a transition, labelled with the input/condition that triggers it.

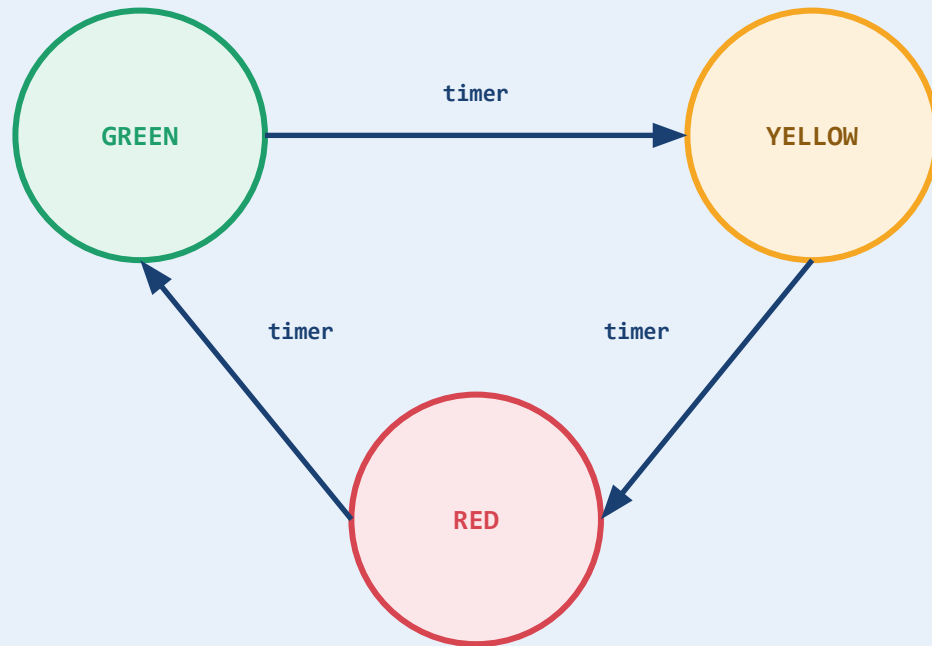
Follow the arrows and you can trace exactly what the machine does for any input sequence.

This diagram IS the design — everything else is just translating it to gates.

The State Diagram

The picture that captures an FSM: circles are states, arrows are transitions.

TRAFFIC LIGHT — A THREE-STATE MOORE MACHINE



HOW TO READ IT

Each circle = one state.

Each arrow = a transition, labelled with the input/condition that triggers it.

Follow the arrows and you can trace exactly what the machine does for any input sequence.

This diagram IS the design — everything else is just translating it to gates.

The State Table

The same FSM, written as a table — the bridge from diagram to hardware.

Current State	Input	Next State	Output
GREEN	expired	YELLOW	Go
YELLOW	expired	RED	Slow
RED	expired	GREEN	Stop
any	not expired	(same)	(hold)

Why a table? It lists every (state, input) → (next state, output). Encode the states as bits, and each column becomes a truth table you minimise with K-maps (L2) into gates.

FSM Design Flow

Six steps from a word problem to a working circuit — always the same recipe.

1**Understand**

List the states and what triggers each transition.

2**State diagram**

Draw circles and arrows — the visual spec.

3**State table**

Tabulate every state/input → next-state/output.

4**Encode states**

Assign a bit pattern to each state (00, 01, 10...).

5**Derive logic**

K-map each next-state & output bit into gates.

6**Build**

Flip-flops for the state register + that logic.

FSMs Run the World

Once you know the pattern, you'll see state machines in everything you build.



The CPU control unit

Fetch → Decode → Execute → Writeback is literally an FSM. It sequences every instruction (Module 4).



Communication protocols

TCP connection states, USB handshakes, traffic-light controllers — all specified as FSMs.



Software & games

UI flows, game AI ('patrol → chase → attack'), regex engines, parsers — FSMs in code, not gates.

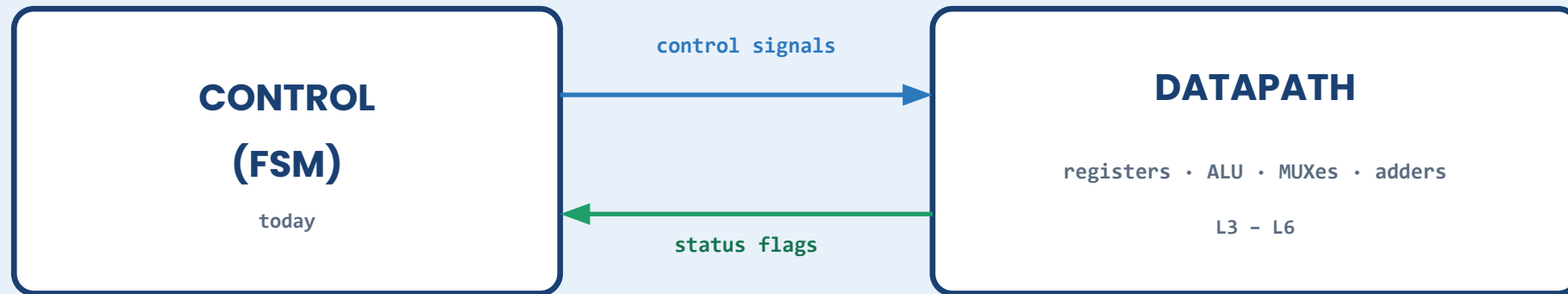


Embedded control

Washing machines, elevators, vending machines, engine controllers — physical FSMs everywhere.

The Big Picture: Datapath + Control

Everything in this course so far now snaps together into a computer.



The control unit is an FSM (today). The datapath is registers + ALU + MUXes (L3-L6). Together they're a CPU.

This is exactly where Module 3 & 4 pick up — you now have every building block a processor needs.

Common Mistakes to Avoid

The slips that cost marks — and cause real design bugs.



Register without load enable

A plain register overwrites every clock. Real ones need a load/hold control, or they can't keep a value.



Moore/Mealy confusion

Moore output = state only (in the circle). Mealy = state + input (on the arrow). Know which you're drawing.



Ripple counter at high speed

The carry ripple makes intermediate counts briefly wrong. Use synchronous counters for anything fast.



Forgetting the reset state

Every FSM needs a defined power-on state, or it starts in a random, possibly illegal state.



Wrong select/state-bit count

N states need $\lceil \log_2 N \rceil$ flip-flops (binary). Miscounting breaks the whole encoding.



Unreachable / dead states

After encoding, unused bit patterns can trap the FSM. Handle them (usually route to reset).

Key Takeaways

Five sentences — from a stored bit to a thinking machine.

- 01** A register is n flip-flops sharing a clock — the CPU's fastest storage; add a load-enable MUX so it holds until told to update.
- 02** A shift register moves bits along a chain each clock — enabling serial↔parallel conversion and free multiply/divide-by-2.
- 03** A counter advances by 1 each clock; synchronous (shared clock) beats ripple (chained) for speed and glitch-free counts.
- 04** A finite state machine — states, transitions, outputs — is the universal model; registers and counters are special cases of it.
- 05** FSM design is a fixed recipe: diagram → table → encode states → K-map the logic → build with a state register + gates.

Practice & Homework

Design, trace, and build state machines.

01

Shift & Count

Trace a 4-bit shift register loading 1011 over 4 clocks. Then draw a 3-bit synchronous counter's Q outputs.

02

FSM from Words

Design a '110' sequence detector: state diagram, state table, and binary encoding.

03

Divide the Clock

How many flip-flops divide a 1 MHz clock down to 1 kHz? Show the divide-by math.

EXIT TICKET

1. How many flip-flops does a mod-12 counter need, and why?
2. In one line: the difference between a Moore and a Mealy machine.
3. Draw the state diagram for a circuit that detects the input sequence '11'.

Rapid-Fire Recap

Three questions — shout the answers.

Q1

A register is built from...

flip-flops

Q2

Synchronous counters share one...

clock

Q3

A Moore output depends only on...

the state

All three instant? You've completed the digital-logic foundation — Module 3 builds the datapath on top of it.

END OF L06 • MODULE 2 COMPLETE

States, and a machine that runs.

NEXT — MODULE 3

The Datapath & Instruction Set — assemble registers, ALU, and control into a processor that actually executes machine instructions.



**Thanks
for
watching!**